

HW3-2 Report: Deep Reinforcement Learning on Proly

Implementation

TODO 6 — Reward Design (`rl_play.py`)

The `RewardManager` computes a composite reward each timestep:

$$r_t = r_{\text{flag}} + r_{\text{dist}} + r_{\text{prox}} + r_{\text{surv}} + r_{\text{respawn}} + r_{\text{time}} + r_{\text{terrain}} + r_{\text{circle}}$$

Flag Capture Reward

$$r_{\text{flag}} = \begin{cases} +200 & \text{if } \text{cp}_t > \text{cp}_{t-1} \\ 0 & \text{otherwise} \end{cases}$$

Returns **+200** when `last_checkpoint_index` increases between frames. The large magnitude makes checkpoint capture the dominant learning signal.

Distance Reward

$$r_{\text{dist}} = \begin{cases} 0 & \text{if } \text{cp}_t \neq \text{cp}_{t-1} \quad \text{(skip on capture frame)} \\ \left(\|\mathbf{p}_{t-1} - \mathbf{p}_{\text{target}}\| - \|\mathbf{p}_t - \mathbf{p}_{\text{target}}\| \right) \times 15 & \text{otherwise} \end{cases}$$

Rewards the agent for reducing distance to the next checkpoint. Skips the frame where a checkpoint is captured to avoid a large negative spike — when captured, `target_position` jumps to the next (farther) flag.

Proximity Bonus

$$r_{\text{prox}} = \begin{cases} 5.0 \times (1 - d_t) & \text{if } d_t < 1.0 \\ 0 & \text{otherwise} \end{cases}$$

where $d_t = \|\mathbf{p}_t - \mathbf{p}_{\text{target}}\|$. Sharp bonus peaking at $d=0$ encourages the agent to commit to the final approach rather than circling near the checkpoint.

Survival Reward (One-Time Death Penalty)

$$r_{\text{surv}} = \begin{cases} -100 & \text{if } \text{hp}_t \leq 0 \text{ and not yet penalized this life} \\ 0 & \text{otherwise} \end{cases}$$

Applied once per death. Earlier versions applied $\$-100$ **every step** while dead, producing episode rewards of $\$-170,000$ and destabilizing the value function. The one-time formulation prevents this.

Respawn Penalty

$$r_{\text{respawn}} = \begin{cases} -5.0 & \text{if agent is respawning} \\ 0 & \text{otherwise} \end{cases}$$

Penalizes the time cost of dying and respawning, teaching the agent that death wastes time even beyond the one-time survival penalty.

Auxiliary Penalties

Time penalty:

$$r_{\text{time}} = -0.1$$

Constant pressure to act rather than idle.

Terrain penalty: For each cell in the 5×5 terrain grid with hazard type $\tau \in \{-1, 1\}$ at relative distance d_{cell} :

$$r_{\text{terrain}} = -\sum_{\text{cells}} \mathbb{1}[d_{\text{cell}} < 0.8] \cdot (0.8 - d_{\text{cell}}) \times 1.0$$

Circling penalty (tiered):

$$r_{\text{circle}} = \begin{cases} -8.0 & \text{if } \Delta_{\text{pos}} < 0.5 \text{ \textit{ over } 50 \textit{ steps (nearly stationary)}} \\ -3.0 & \text{if } \Delta_{\text{pos}} < 1.5 \text{ \textit{ over } 50 \textit{ steps (slow circling)}} \\ 0 & \text{otherwise} \end{cases}$$

where $\Delta_{\text{pos}} = \|\mathbf{p}_t - \mathbf{p}_{t-50}\|$.

Training Setup

The agent uses SB3's PPO (**MlpPolicy**, [64, 64] Tanh, **n_steps=2048**) integrated with the MLGame3D game loop. Because mlgame3d drives environment stepping externally, rollout data is collected manually via **_add_to_rollout_buffer** and policy updates are triggered via **_update_policy** when the buffer fills. CPU inference is used to avoid timing out at 10× time scale.

Training Phases

Phase	Description	Hyperparameters
1. Initial training	Fresh training across maps 0–4	lr=3e-4, clip=0.2, ent=0
2. Multi-map generalization	Map shuffling for robustness	lr=3e-4, clip=0.2, ent=0
3. Fine-tuning for map 4	Cycled maps 1–3 (200 eps) + map 4 (500 eps ×2), 3 cycles	lr=1e-4, clip=0.15, ent=0.005

Fine-tuning used reduced learning rate and tighter clipping to avoid catastrophic forgetting on maps 1–3, with small entropy to encourage exploring alternative paths on map 4.

Results

Map	Flags (best of 3)	Completion Time
Map 0 (Island)	10/10	~21–27s

Map	Flags (best of 3)	Completion Time
Map 1 (S-shaped)	10/10	~27–43s
Map 2 (V-shaped)	10/10	~21–36s
Map 3 (2-hole)	10/10	~27–33s
Map 4 (Pothole)	10/10	~28–115s

All maps achieve 10/10 flag capture within the 120-second limit.

Failure Analysis and Fix (Map 4)

Root Cause

Map 4 ("Pothole Island") has **water (\$\tau = -1\$) in the middle of the island** as potholes, unlike maps 1–3 where water only appears at borders. The agent initially scored 0/10 on map 4.

Failed Approach: Stronger Water Penalty

The intuitive fix — increasing the water terrain penalty — backfired. The agent learned to avoid water but became **too cautious**. On map 4, narrow corridors (1-grid-wide safe passages between water potholes and the island border) are the only viable paths. With a strong water penalty, the expected reward for entering the corridor was:

$$\mathbb{E}[r_{\text{corridor}}] \approx r_{\text{dist}} + r_{\text{terrain}}^{\text{high}} < 0$$

The agent preferred circling in safe open areas ($r_{\text{circle}} = -1.0$, old value) over risking the corridor, because $|-1.0| < |r_{\text{terrain}}^{\text{high}}|$.

Successful Approach: Reward Rebalancing

Instead of making water scarier, we made **circling more expensive** and **forward progress more rewarding**. The key insight is the cost-benefit inequality the agent evaluates at the corridor entrance:

$$\underbrace{r_{\text{dist}} \times 15}_{\text{approach pull}} + \underbrace{r_{\text{flag}}}_{\text{+200 at end}} \gg \underbrace{r_{\text{terrain}}}_{\text{mild water penalty}} + \underbrace{r_{\text{circle}}}_{\text{-3 to -8 if stuck}}$$

The three changes that made the agent commit to narrow paths:

1. **Tiered circling penalty** (\$-1 \to -3/-8\$): Made the "safe" circling option much more expensive
2. **Distance multiplier** (\$10 \to 15\$): Increased the per-step incentive to move toward the flag
3. **One-time death penalty**: Removed the catastrophic $-\$100 \times T_{\text{remaining}}$ noise, so the value function could learn that dying once is recoverable but circling forever is not

The result: at the corridor entrance, the expected cumulative reward for pushing through (brief terrain penalty $\rightarrow$ +200 flag capture) now clearly dominates the alternative of circling indefinitely (\$-3\$ to \$-8\$ per step for hundreds of steps).